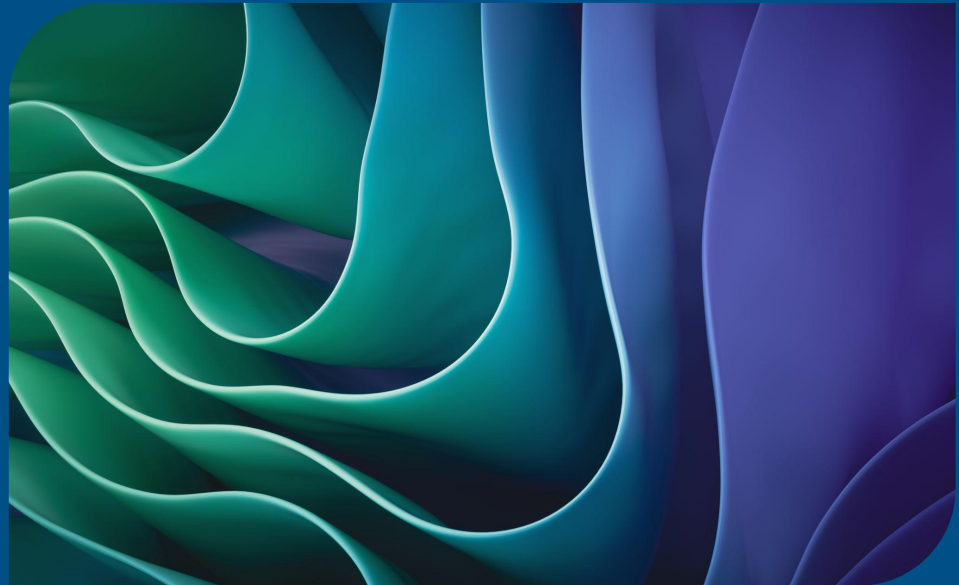


Non-Functional Requirements (NFRs) & Constraint

Enable engineers to **reason from business outcomes to architectural decisions** using NFRs and constraints – rather than copying reference architectures or patterns blindly.



What Are Requirements, Really?

Requirements define *what* success looks like, not *how* to build it.

1. **Functional Requirements (FRs):** What the system does
2. **Non-Functional Requirements (NFRs):** How well the system does it
3. **Constraints:** What the system *must* or *must not* do

What Are Requirements, Really?

Requirements define *what* success looks like, not *how* to build it.

1. **Functional Requirements (FRs):** What the system does
2. **Non-Functional Requirements (NFRs):** How well the system does it
3. **Constraints:** What the system *must* or *must not* do

What Are Requirements, Really?

Requirements define *what* success looks like, not *how* to build it.

1. **Functional Requirements (FRs):** What the system does
2. **Non-Functional Requirements (NFRs):** How well the system does it

Functional vs Non-Functional Requirements

Functional Examples

- User can place a Transfer
- System calculates tax

Functional vs Non-Functional Requirements

Functional Examples

- User can place a Transfer
- System calculates tax

Anti-Pattern

The system should be:

- Fast
- Scalable
- secure

Functional vs Non-Functional Requirements

Functional Examples

- User can place a Transfer
- System calculates tax

Standard Practice

NFRs must be *measurable, testable, and time-bound*

Anti-Pattern

The system should be:

- Fast
- Scalable
- secure

Functional vs Non-Functional Requirements

Functional Examples

- User can place a Transfer
- System calculates tax

Anti-Pattern

The system should be:

- Fast
- Scalable
- secure

Standard Practice

NFRs must be *measurable, testable, and time-bound*

Non-Functional Examples

- Transfer placement completes in < 200 ms (**p95**)
- Internal transfers completes in < 1 Seconds
- System handles 10k concurrent users in Peak-hour
- Data is encrypted at rest

Question: What is **p95** ?

Categories of Non-Functional Requirements

- Performance (latency, throughput)
- Availability & Reliability
- Scalability
- Security & Privacy
- Maintainability & Operability
- Cost & Efficiency
- Compliance & Legal

Latency, Throughput, and Consistency

- **Latency**: time to serve a request
- **Throughput**: requests per unit time
- **Consistency**: correctness/freshness of data

Availability Targets (99.9 vs 99.99)

What the Numbers Actually Mean (**Annual Downtime: 365 days**)

- 99% $\approx$ 87.6 hours (3 days 15 hours and 36 minutes !!)
- 99.9% $\approx$ 8.7 hours
- 99.99% $\approx$ 52 minutes

Implications

- Redundancy (Data, Service instance etc)
- Automated failover
- Operational maturity etc.

Why Availability Is Not Free

Anti-Pattern

- Let's just aim for four nines (99.99%)

Concerns:

- More replicas → more coordination
- More regions → more latency and complexity
- Higher availability → higher cost and operational burden

Data Retention & Auditability

Key Questions

- What data(category??) must be retained?
- For how long?
- Who can access or delete it?

Trade-offs:

- Storage cost vs legal risk
- Immutability vs user privacy

From NFRs to Architecture

Business Need

- Global users
- Checkout must be fast
- Occasional inconsistency acceptable

Derived NFR:

- p95 latency < 300 ms
- Availability $\geq 99.9\%$

Architecture Choices:

- Regional read replicas (Microservice, CQRS)
- Eventual consistency (Event-driven architecture)
- CDN (for static files)

Common Failure Modes

Common practice

- NFRs not written down
- NFRs discovered after launch
- Architecture copied from another company without matching context

Standard Practice:

- Treat NFRs as first-class requirements

Key Takeaways

- **NFRs encode business priorities**
- **Trade-offs are unavoidable—make them explicit**
- **Good engineers explain why, not just what**
- **Avoid HDA (Hype-driven Architecture)**
- **Avoid HDD (Hype-driven Development)**

Standard Practice:

- **Treat NFRs as first-class requirements**
- **Justify your architectural decisions**
- **Justify your techstacks selection**

Practical Exercise

Scenario: Think about the simplest module of your team project

Define:

- Functional requirements
- Non-functional requirements
- Logical justification of Architectural decision
- Logical justification of techstack selection

Thank you ...